

Uygulama Planı: Java 27 Öğrenim Müfredatı

Aşama: Specify → Plan → Tasks → Implement dizisinin 2. aşaması (Plan) **Spesifikasyon:** [SPEC.md](#)

Tarih: 2026-09-23

Doğrulanmış Ön Koşullar

Denetim	Sonuç
JDK 27 kurulu	<code>java version "27" 2026-09-15</code> , derleme 27+35-2325
JDK 27 varsayılan	Evet, <code>/Library/Java/JavaVirtualMachines/jdk-27.jdk</code>
JEP 512 kompakt kaynak dosyaları bayraksız çalışıyor	Doğrulandı, <code>java HelloWorld.java</code> çıktısı <code>Hello, World!</code> , çıkış 0
<code>gh</code> CLI yetkili	Evet, hesap <code>aliturgutbozkurt</code> , <code>repo</code> kapsamı mevcut

Modül 01'in giriş yolu varsayılmadı, gerçek araç zincirinde kanıtlandı.

Açık Sorularda Alınan Kararlar

Spesifikasyondaki 2 ve 3 numaralı açık sorular açıkça cevaplanmadı, dolayısıyla spesifikasyonun kendi önerileri geçerli:

- Test (Modül 21):** sıfır bağımlılık. `-ea` ile çıplak `assert` artı elle yazılmış bir mikro çatı, ve JUnit'in ne kattığını, gerçek projelerin onu neden kullandığını açıklayan bir bölüm. Bağımlılık yasağı korunuyor.
- Preview özellikler:** yalnızca Modül 22'de, JEP numarası ve preview turuyla açıkça etiketlenmiş, `--enable-preview --source 27` arkasında çalıştırılmış. Kaldırılmadılar, çünkü bir öğrenci kararlı ile preview arasındaki farkı bilmeli, ama karantinaya alındılar ki daha öncesindeki hiçbir şey onlara bağlı olmasın.

İkisinden biri değişmeli ise söyleyin.

Bileşenler

Bileşen	Sorumluluk	Bağımlılığı
scaffold	Depo iskeleti, doğrulama scripti, gitignore, README taslağı	—
template	Modül formatının referans uygulaması olarak Modül 01	scaffold
core-language	Modül 02–10, Java'ya yeni gelenin önce ihtiyaç duyduğu sözdizimi ve nesne modeli	template
type-system	Modül 11–12, jenerikler ve koleksiyonlar	core-language
functional	Modül 13–15, lambda'lar, stream'ler, Optional	type-system
platform	Modül 16–21, IO, standart kütüphane, eşzamanlılık, modüller, test	functional
release-literacy	Modül 22, 26 ve 27'nin ne değiştirdiği, JEP'ler, preview ile kararlı	platform
integration	Kök README içindekiler tablosu, tam depo doğrulama geçişi	hepsi

Kurulum sırası: scaffold → template → core-language → type-system → functional → platform → release-literacy → integration

Bu Sıra Neden

Sıralama teknik değil pedagojik. Şablon var olduktan sonra modül yazımı ilkesel olarak herhangi bir sırada olabilir, ama her modülün metni öğrencinin öncekileri okuduğunu varsayıyor. Sırayla yazmak çapraz göndermeleri dürist tutan şey; örneğin Modül 14'ün stream'lerde Modül 13'te tanıtilan fonksiyonel arayüzlere geri atıf yapması.

Not edilmeye değer iki istisna:

- scaffold önce ve eksiksiz gelmeli. Doğrulama scripti sonraki her örneğin çalıştığını kanıtlayan şey, dolayısıyla o var olmadan modül yazmak doğrulanmamış içerik biriktirmek demek.
- template yalnızca ilk modül değil, bir kapı. Modül 01 yedi bölümlük README yapısını, yorum stilini, ödev biçimini ve çözüm düzenini sabitliyor. Yanlış yapmak 21 modülü yeniden çalışmak demek, dolayısıyla Modül 02 başlamadan önce inceleniyor.

Riskler ve Önlemler

Risk	Etki	Önlem
Preview özellikler kesinleşmeden değişir	Modül 22 değişen bir şeyi öğretir	Modül 22'ye karantina, her preview açıkça etiketli, JEP numarası ve preview turu belirtilerek okuyucunun güncel durumu denetleyebilmesi sağlanır
Uydurulmuş sürüm iddiaları	Müfredat yanlışları kendinden emin biçimde öğretir	Spesifikasyon sınırı bunu yasaklıyor. Her JDK iddiası openjdk.org'a dayanıyor. Zaten uygulandı: JDK 27 JEP tablosu hafızadan değil sürüm sayfasından geldi
22 modül boyunca kapsam yorgunluğu	Proje yarım kalır	Issue başına bir modül, her biri bağımsız olarak teslim edilebilir ve doğrulanabilir. Doğrulanmış 11 modüllük yarım bir müfredat bile öğretir
Örnekler gelecekteki bir JDK'da sessizce bozulur	Çürüme	<code>verify-examples.sh</code> CI'da çalıştırılabilir ve her örneği ve çözümü kapsar
Modüller yazıldıktan sonra şablon değişimi	Pahalı yeniden çalışma	Modül 01, Modül 02 öncesinde açık bir inceleme kapısı
Öğrencinin JDK sürümü 27'den farklı	Örnekler kafa karıştırıcı biçimde başarısız olur	Kök README gerekli JDK'yı belirtiyor, doğrulama scripti <code>java -version</code> denetliyor ve uyumsuzlukta gürültülü biçimde başarısız oluyor

Paralel ve Ardışık

- **Ardışık:** scaffold → template → diğer her şey. Ayrıca içerik bağımlılığı nedeniyle Modül 13'ten önce 14, ve 08'den önce 09.
- **Paralleleştirilebilir:** `platform` içinde Modül 16, 17, 20 ve 21 birbirine içerik bağımlılığı taşıyor, herhangi bir sırada ya da farklı kişilerce aynı anda yazılabilir.

Doğrulama Kontrol Noktaları

Sonrasında	Denetim
scaffold	<code>./scripts/verify-examples.sh</code> boş ağaçta 0 ile çıkıyor ve bilerek bozuk bir dosya verildiğinde gürültülü biçimde başarısız oluyor
template	Modül 01 doğrulamadan geçiyor ve README'si yedi bölümü de içeriyor
her modül	<code>./scripts/verify-examples.sh modules/<id></code> 0 ile çıkıyor, ödev metni mevcut, referans çözüm temiz çalışıyor
integration	Tam depo doğrulaması 0 ile çıkıyor, ve sekiz spesifikasyon başarı kriterinin hepsi tek tek işaretleniyor